2.3 Using `redirect_to`

Another way to handle returning responses to an HTTP request is with `redirect_to`. As you've seen, `render` tells Rails which view (or other asset) to use in constructing a response.

The `redirect_to` method does something completely different: it tells the browser to send a new request for a different URL. For example, you could redirect from wherever you are in your code to the index of photos in your application with this call:

```
redirect_to photos_url
```

You can use `redirect_to` with any arguments that you could use with `link_to` or `url_for`. There's also a special redirect that sends the user back to the page they just came from:

```
redirect_to :back
```

2.3.1 Getting a Different Redirect Status Code

Rails uses HTTP status code 302, a temporary redirect, when you call `redirect_to`. If you'd like to use a different status code, perhaps 301, a permanent redirect, you can use the `:status` option:

```
redirect_to photos_path, status: 301
```

Just like the `:status` option for `render`, `:status` for `redirect_to` accepts both numeric and symbolic header designations.

2.3.2 The Difference Between `render` and `redirect_to`

Sometimes inexperienced developers think of `redirect_to` as a sort of `goto` command, moving execution from one place to another in your Rails code. This is *not* correct. Your code stops running and waits for a new request for the browser. It just happens that you've told the browser what request it should make next, by sending back an HTTP 302 status code.

Consider these actions to see the difference:

```
def index
  @books = Book.all
end

def show
  @book = Book.find_by(id: params[:id])
  if @book.nil?
    render action: "index"
  end
end
```

With the code in this form, there will likely be a problem if the `@book` variable is `nil`. Remember, a `render :action` doesn't run any code in the target action, so nothing will set up the `@books` variable that the `index` view will probably require. One way to fix this is to redirect instead of rendering:

```
def index
  @books = Book.all
end

def show
  @book = Book.find_by(id: params[:id])
  if @book.nil?
    redirect_to action: :index
  end
end
```

With this code, the browser will make a fresh request for the index page, the code in the `index`method will run, and all will be well.

The only downside to this code is that it requires a round trip to the browser: the browser requested the show action with `/books/1` and the controller finds that there are no books, so the controller sends out a 302 redirect response to the browser telling it to go to `/books/`, the browser complies and sends a new request back to the controller asking now for the `index` action, the controller then gets all the books in the